
Phase 2: Feasibility Study for Intelligent, AI-Powered Circular Campus Resource Exchange and Asset Life Cycle Management System

Version 1.0

Prepared by Group 6

Abi Ali Murtaza Ghadiyali — 100066653
Ahmed Mohamed Moustafa Ghandour — 100066431
Abdulrahman Feras Junaid — 100066011
Muhammad Awais Afzal Mirza — 100066617

COSC 336: Introduction to Software Engineering

Lab Instructors: Basima Kurungadan and Natnael Takele
Course Professor: Dr. Jamal Bentahar

Fall 2026
20th September 2026

Table of Contents

1. Project Scope and Objectives.....	1
1.1 Purpose of the Study	1
1.2 Project Scope.....	1
1.3 Assessment Objectives and Reference Scenario	1
2. Stakeholders and Operational Feasibility.....	2
2.1 Stakeholder Needs and Responsibilities	2
2.2 Workflow Adoption and Organizational Readiness.....	2
3. Market Need and Existing Alternatives.....	3
3.1 Need and Demand.....	3
3.2 Existing Approaches and Alternatives	3
4. Technical, Data, and AI Feasibility	4
4.1 Technical Alternatives and Proposed Approach	4
4.2 Data Availability and Quality	5
4.3 AI Capability and Integration	6
4.4 Skills, Resources, and Validation Needs.....	6
5. Financial and Economic Feasibility.....	7
5.1 Cost Basis and Estimates	7
Hosting and AI costs	7
Development and maintenance costs	8
5.2 Benefits, ROI, and Sensitivity.....	8
6. Legal, Ethical, Privacy, and Policy Feasibility	9
6.1 Legal, Licensing, and Institutional Constraints.....	9
6.2 Privacy, Security, and Responsible AI.....	10
7. Risk Assessment and Contingencies.....	10
8. Project Schedule and Delivery Feasibility	11
9. Recommendation and Action Plan	12

Revision history

Name	Date	Reason For Changes	Version

1. Project Scope and Objectives

1.1 Purpose of the Study

This feasibility study assesses whether Group 6 can develop the proposed AI-powered campus resource-exchange and asset life-cycle management prototype within the semester. It builds on the scope and initial requirements established in the Phase 1 report.

The study examines technical and data requirements, operational adoption, costs and expected benefits, legal and ethical constraints, risks, and delivery resources. It compares practical alternatives and identifies the conditions needed to proceed. Conclusions about the semester prototype are distinguished from those about a future university deployment, which would require further operational evidence and institutional approval.

1.2 Project Scope

The proposed system will support internal resource exchange and asset life-cycle management in universities. Its scope includes asset registration and publication, departmental requests, search, reservations, approvals, transfers, maintenance, repair, refurbishment, and end-of-life records. AI-assisted functions will support classification, semantic matching, sustainable-action recommendations, conversational assistance, and reporting.

The semester deliverable is a working prototype using synthetic data. It will maintain its own records without requiring integration with KU or other institutional systems. Physical collection, storage, inspection, and repair remain staff responsibilities; the software will coordinate these activities and record their outcomes. Complete procurement, accounting, transport, and warehouse-management systems are outside scope.

Feasibility depends on access to suitable development tools, hosting, data storage, AI inference, and sufficient team time. This study will assess those dependencies and compare implementation options. The prototype will demonstrate the proposed workflows rather than replace an existing university system.

1.3 Assessment Objectives and Reference Scenario

The study will assess whether the required functionality can be delivered with suitable technology, manageable learning and integration effort, and realistic operating costs. It will also examine whether the proposed workflows are practical for participating staff and whether expected benefits justify the resources and risks involved.

For consistent comparison, the reference scenario assumes 20 participating academic departments and service units, 200 authorized users, approximately 40 daily active users, and five simultaneous sessions. The initial operating catalog contains 5,000 asset records, including 500 available listings. Records represent either individually tracked assets or quantities of interchangeable items with shared characteristics; the catalog does not represent every physical item owned by a university.

Monthly activity is estimated at 200 new or substantially updated records, 100 departmental requests, 40 completed transfers, 1,600 semantic searches, and 400 assistant turns. Primary

photograph and document storage is estimated at 12 GB, excluding database storage, indexes, logs, and backups. These figures are planning assumptions rather than observed university data.

Lower and higher usage cases will test how changes in activity and data volume affect the conclusions. Prototype evaluation will use one coherent synthetic catalog, with curated requests and expected outcomes for correctness and AI assessment, and larger volume tests for performance. Financial estimates will distinguish semester prototype expenditure from an illustrative 12-month university operating scenario.

2. Stakeholders and Operational Feasibility

2.1 Stakeholder Needs and Responsibilities

The stakeholders identified in Phase 1 remain relevant, but operational feasibility depends on whether their responsibilities can fit existing work practices.

Requesting departments need suitable resources, reliable information, simple processes and clear request progress.

Supplying departments and asset custodians need a low-effort way to publish surplus items, maintain their records, authorize release, and complete a documented handover. Participation is unlikely to remain effective if updating listings creates excessive work or responsibility remains unclear after collection.

Administrators, procurement, and finance personnel need controlled approvals, traceable asset changes, and credible cost information.

Maintenance and facilities personnel need accurate locations, condition records, and clear collection or service assignments.

Sustainability officers need activity records and transparent assumptions behind reported benefits.

System administrators would manage access and technical operation.

Group 6 is responsible for developing and evaluating the prototype, while the professor and lab instructors review its academic deliverables. A future university deployment would additionally require an identified operational owner and continuing support; the student development team does not establish these institutional commitments.

2.2 Workflow Adoption and Organizational Readiness

The proposed workflow is operationally realistic because it supports recognizable activities: listing resources, requesting them, authorizing release, and confirming receipt. Adoption nevertheless depends on keeping these activities convenient and easy to use. Reusing stored information, displaying the next required action, and assigning clear approval responsibilities can reduce repeated entry and follow-up communication. Brief role-specific guidance would support initial use.

Accurate records also require ongoing ownership. Participating departments would need to confirm availability and report condition changes, while collection and storage arrangements must be supported by actual staff and capacity. AI recommendations cannot compensate for stale records or guarantee a physical handover.

Operational feasibility is therefore conditional on an identified process owner, participating departments, and handling arrangements. The prototype can test workflow clarity through role-based walkthroughs, but institutional adoption and staffing commitments would require validation before deployment.

3. Market Need and Existing Alternatives

3.1 Need and Demand

The project addresses a practical problem: a department may purchase equipment without knowing that another department already has a suitable unused item. Making that equipment easier to find and obtain could avoid unnecessary purchases and reduce waste.

Universities already support internal reuse. The University of Cambridge provides WARPit to help departments reuse equipment, furniture, and other items. KU policy guidance retrieved through KUBee similarly describes equipment-discovery services, inventory and fixed-asset records, and approved transfer procedures involving departmental staff, Procurement/Materials Management, and Finance.

These processes suggest that discovering an item and obtaining it are separate activities involving several responsibilities and records. The proposed system aims to connect discovery, requests, approvals, and asset history through a clear and simpler workflow, with AI helping users classify resources and identify suitable matches.

The presence of existing services supports the relevance of the problem, but does not by itself establish demand for another application. The prototype will therefore examine whether the proposed approach makes reuse easier for both requesting and supplying departments.

Sources: University of Cambridge, [“Waste and circular economy”](#); KU guidance retrieved through KUBee, citing Khazna Procedures (July 2025), KUConnect Links (2025), and policies KUP 6500, KUP 6510, and KUP 6600.

3.2 Existing Approaches and Alternatives

Continue with existing systems and coordination: Universities can retain their asset registers, request forms, and approval procedures, using staff communication to arrange reuse. This avoids introducing another application and preserves established responsibilities. However, where information is spread across systems, users may still need to contact several people to confirm availability and complete a transfer.

Adopt or extend an existing reuse platform: Services such as WARPit already support the redistribution of university resources. An institution should therefore consider whether an existing platform can meet its needs before commissioning new software. This could reduce custom development, although suitability would depend on licensing, integration, permissions, and support for the required asset-management workflows. Those details would need a product-specific assessment.

Develop the proposed integrated prototype. This approach allows the team to demonstrate the exchange, life-cycle, and AI capabilities required by the project brief within one workflow. It

offers control over the user experience and business rules, but introduces development, testing, and maintenance responsibilities.

For this course, developing the prototype is the required route. For a future university deployment, the decision should depend on whether its demonstrated benefits justify building and maintaining it instead of improving existing systems or adopting an available solution.

4. Technical, Data, and AI Feasibility

4.1 Technical Alternatives and Proposed Approach

Three approaches were considered: a managed backend with a browser-based frontend; a Cloudflare-native application using Workers, D1, Vectorize, and R2; and a conventional Django application with PostgreSQL. Each provides a credible development path, but distributes implementation and maintenance work differently.

The recommended stack is:

- **Frontend:** React with TypeScript and Vite, hosted through Cloudflare Workers Static Assets.
- **Backend services:** Supabase authentication, file storage, and server-side functions.
- **Database and semantic retrieval:** PostgreSQL with pgvector, keeping asset records and their searchable vectors in the same database.

This arrangement supplies the main backend facilities without requiring the team to operate a separate application server. The alternatives offer different advantages:

- **Cloudflare-native:** lower published service fees, but more coordination between authentication, relational records, and a separate vector index.
- **Django with PostgreSQL:** integrated backend facilities and a conventional Python environment, but still requires application hosting and purpose-built user screens.

The managed approach is preferred because it lets the team concentrate on project-specific workflows. Database policies and transactional operations must enforce authorization and prevent competing reservations from allocating the same stock. Server-side functions will handle AI credentials and external calls; the browser will not receive secret keys. The proposed workload provides a starting point for sizing, while actual response times and resource limits must be checked during implementation.

Sources: Supabase [row-level security](#) and [database functions](#); [pgvector](#); Cloudflare [Workers Static Assets](#) and [D1](#); [Django administration](#).

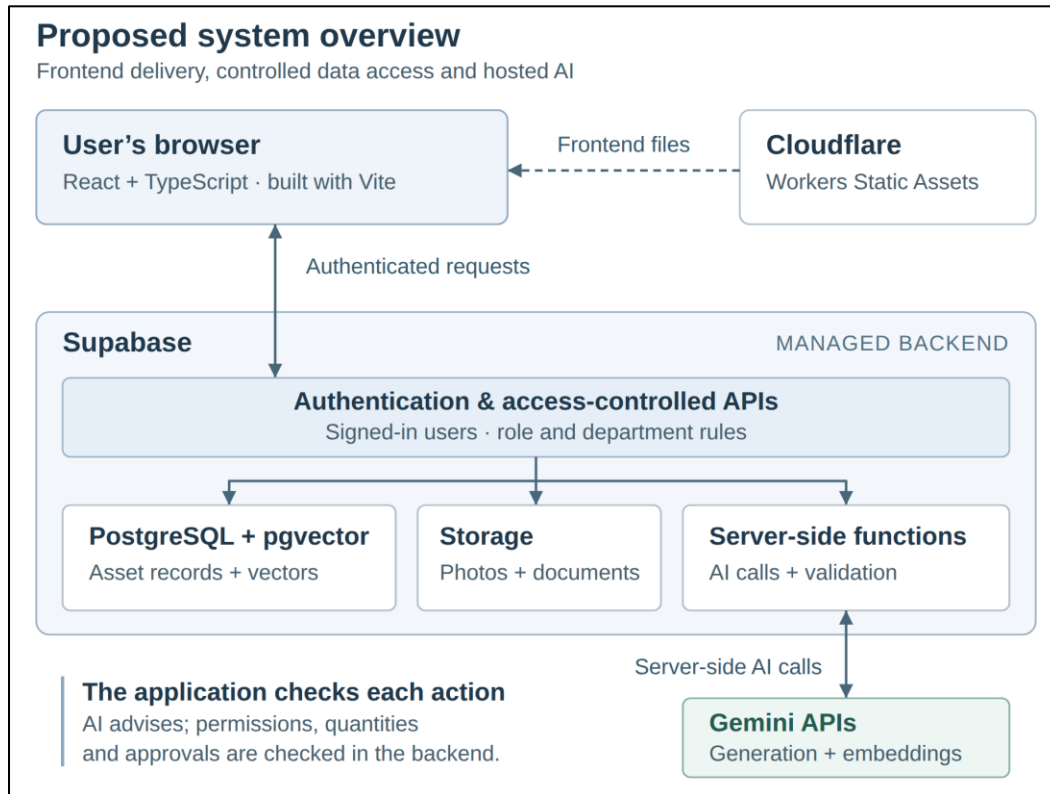


Figure 1. Proposed system overview

4.2 Data Availability and Quality

No university database will be supplied, so the prototype will use a synthetic catalog representing participating departments already using the system. This makes data preparation achievable without depending on institutional access, but the records must be consistent enough to support meaningful tests.

The dataset will include:

- **Department-relevant inventory:** individually tracked equipment and suitable quantity-based records, with varied specifications, conditions, locations, and availability.
- **Connected operational records:** users, requests, reservations, transfers, and maintenance history linked to the relevant assets.
- **Representative photographs and documents:** selected examples for testing uploads and AI classification, rather than manufacturing the full estimated storage volume.

Curated test requests will have expected outcomes defined before evaluation, including suitable matches, incompatible alternatives, missing information, insufficient quantities, and no-match cases. These requests will be tested against the shared catalog, with some held back while matching behaviour is refined. Additional generated records will support larger-volume performance tests.

Validation will check required fields, valid relationships, quantities, and consistent workflow states. A reproducible starting dataset will allow tests to be repeated after records are added or

modified. Real personal or confidential information is unnecessary for these tests. Synthetic data can demonstrate functionality and expose defects, but cannot establish matching accuracy or adoption across actual universities.

4.3 AI Capability and Integration

The proposed approach uses Google's Gemini API rather than training or hosting models ourselves. Two models serve different purposes:

- **Gemini 3.5 Flash-Lite:** processes text, images, and PDFs to suggest asset details, answer questions, recommend actions, and produce report summaries. It can return information in a defined format for the application to check.
- **Gemini Embedding 2:** converts asset descriptions and search requests into numerical representations of their meaning, called embeddings. These are stored in PostgreSQL and searched using pgvector to find potentially relevant items.

Cloudflare Workers AI is a credible alternative, offering image-capable models, embeddings, and document conversion. Gemini is preferred because its documented PDF handling and formatted responses provide a simpler starting point for the required features. Actual usefulness must still be tested with the project's sample data.

AI requests will pass through server-side functions that protect API keys and restrict access to information the user is allowed to see. The application will check required fields, suitability, available quantities, and permissions before acting on model output. Financial totals will be calculated by ordinary code and supplied to the model for explanation.

Users will review suggested asset details and prepared requests before saving them. If AI assistance fails, conventional search, forms, and approval workflows will remain available. Testing will compare AI-assisted matching with ordinary search and check whether generated answers and reports agree with the underlying records.

Sources: Google [Gemini 3.5 Flash-Lite capabilities](#) and [embeddings](#); Cloudflare Workers AI [model capabilities](#), [document conversion](#), and [JSON Mode](#).

4.4 Skills, Resources, and Validation Needs

The project requires skills in frontend development, SQL, access control, API integration, testing, and deployment. The work also includes database design, AI-service configuration, and documentation of the system's setup and operation.

Development can use team computers, GitHub, and the proposed hosted services. A dedicated GPU is not required because model processing will take place through external APIs. Internet access, service accounts, usage limits, and an agreed spending allowance will need to be arranged.

Before building all features, a small working example should confirm that:

- Users can access only the records and actions permitted by their roles.
- Two competing requests cannot reserve the same remaining item.
- The AI services can process representative text, images, and documents and return usable results.

- Search meets the provisional Phase 1 performance target under the declared test conditions.
- Both database records and uploaded files can be backed up and restored.

These checks target the main uncertainties early. Failed checks would lead to changes in the implementation approach before the team builds further features around it.

5. Financial and Economic Feasibility

5.1 Cost Basis and Estimates

Costs use the reference scenario in Section 1.3: 200 authorized users, 5,000 asset records, approximately 12 GB of uploaded files, and the stated monthly request and AI activity. Prices are expressed in US dollars and were checked in September 2026. Service prices are published figures; usage, work hours, and labour values are planning assumptions.

Hosting and AI costs

The frontend will use Cloudflare Workers Static Assets, which provides free storage and delivery of directly served frontend files under its published terms. Static files have a 25 MiB per-file limit. Requests that execute Worker code are subject to separate request and processing limits; no such backend execution is included in the \$0 frontend estimate.

Uploaded asset photographs and documents will be stored in Supabase, not treated as free frontend files. Supabase Pro starts at \$25 per month and includes 100 GB of file storage, covering the estimated 12 GB. Its included user allowance also exceeds the assumed 200 accounts, although response times and processing capacity still require testing.

The AI estimate assumes the following monthly use:

- **400 assistant turns**, including help preparing requests.
- **100 asset classifications**, representing half of the 200 new or substantially updated records.
- **100 sustainable-action recommendations**, an additional planning assumption.
- **21 report summaries**: one per participating unit and one university-wide summary.
- **1,600 search-query embeddings** and embeddings for 200 updated asset records.

Assumed request lengths, including a 10% allowance for retries and variation, total approximately 1.95 million generation-input tokens, 0.30 million output tokens, and 0.16 million embedding tokens. Tokens are the units providers use to measure model input and output. At the selected Gemini models' published rates, this gives approximately \$1.38 per month. A \$5 AI allowance is used because actual conversation lengths and image/document processing have not yet been measured.

Item	Monthly planning cost
Cloudflare static frontend—direct file delivery only	\$0
Supabase Pro	\$25
Gemini generation and embeddings	\$5 allowance
Additional usage allowance	\$5
Total	\$35

Three billed prototype months would cost approximately \$105, assuming existing computers and internet access. Initial text embedding of 5,000 records, averaging 400 tokens each, would add approximately \$0.40, accommodated within the usage allowance.

Development and maintenance costs

A university prototype must also account for the work involved. The initial estimate allocates 60 hours to planning and design, 160 to core application development, 80 to AI integration, and 100 to testing, deployment, and documentation. These 400 hours are a starting work estimate based on the required features and use of managed services; the schedule assessment will examine whether they are achievable.

Valuing this effort at an assumed \$25 per hour gives \$10,000 in development cost. Allowing eight support hours per month for updates, fixes, and routine checks adds \$2,400 annually. Neither the hourly rate nor support allocation is a supplier quotation.

Including twelve months of services, the first-year estimate is \$12,820, or \$14,820 with a 20% development effort reserve. Student time is not a cash payment for the course prototype, but is included when estimating the economic cost of a university pilot.

Taxes, new hardware, institutional integrations, formal security assessments, and stronger backup arrangements are excluded. These would require further budgeting before a wider deployment.

Sources: [Cloudflare Workers Static Assets billing](#); [Supabase pricing](#); [Google Gemini API pricing](#).
Workload, token usage, effort, and labour values are study assumptions.

5.2 Benefits, ROI, and Sensitivity

The main financial benefit is avoiding purchases by reusing suitable existing resources. Savings should count only where the proposed system enables reuse that would not otherwise occur through existing procedures.

The reference workload gives 40 completed transfers per month, or 480 annually. For an illustrative base case, assume half replace one planned purchase each, averaging \$150 per purchase. This value represents ordinary furniture and office equipment rather than expensive laboratory instruments. Allow an average \$25 per completed transfer for handling and minor repair. These are study assumptions, not measured university figures; multi-item transfers would require quantity specific calculations.

The base case therefore produces \$36,000 in avoided purchases and \$12,000 in transfer related costs annually. Adding the \$12,820 first-year system cost gives total costs of \$24,820 and a net benefit of \$11,180.

$$\text{First-year ROI} = \frac{\text{Avoided purchase costs} - \text{Total costs}}{\text{Total costs}} \times 100\%$$

At purchase avoidance shares of 25%, 50%, and 75%, annual avoided purchase costs are \$18,000, \$36,000, and \$54,000, giving first-year ROI values of -27.5%, 45.0%, and 117.6%, respectively. Figure 2 shows the corresponding net benefits and break-even point.

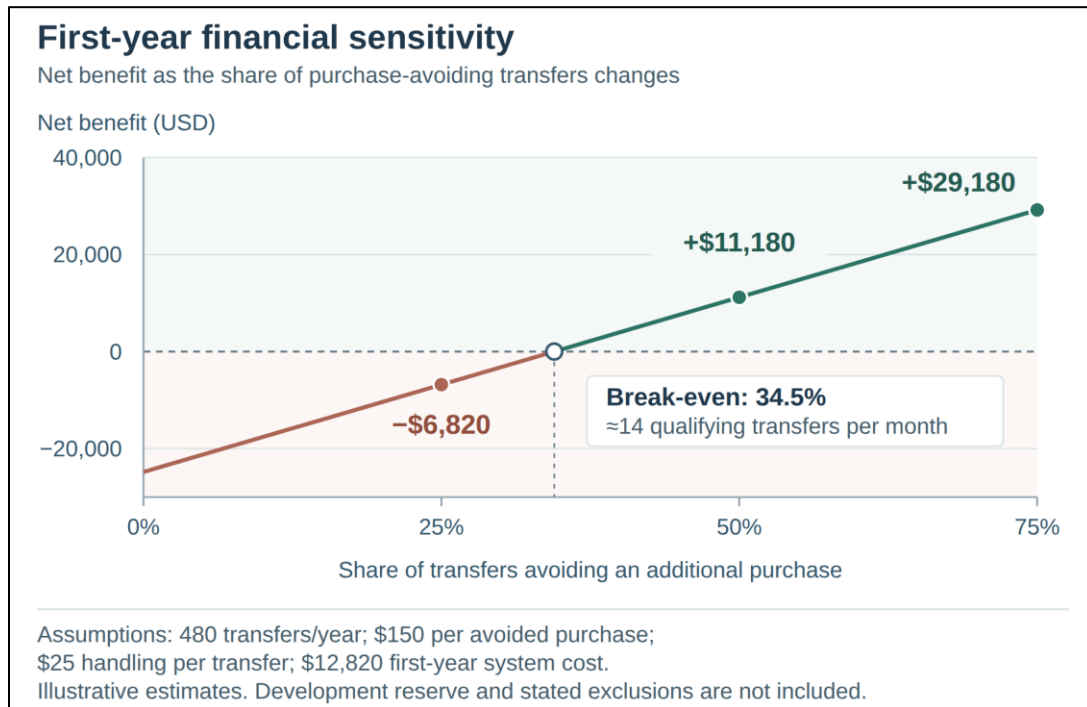


Figure 2. First-year net benefit under alternative purchase-avoidance scenarios.

This comparison holds transfer volume and unit costs constant to show how strongly the result depends on additional purchase avoidance. Including the development reserve raises total base-case costs to \$26,820, reducing ROI to approximately 34%. Without that reserve, break-even requires about 14 of the 40 monthly transfers to replace purchases at the assumed value.

The scenario suggests potential financial value, but not a guaranteed return. Lower replacement values, higher repair costs, slower adoption, or greater development effort could reverse the result. Reduced waste, longer asset life, and easier coordination provide further potential benefits, but are not assigned monetary values here.

6. Legal, Ethical, Privacy, and Policy Feasibility

6.1 Legal, Licensing, and Institutional Constraints

The prototype will use synthetic records and test accounts. Sample photographs and documents must be team-created or used with permission. This reduces dependence on institutional data, but does not remove the need to follow software licences and service terms.

Each selected dependency must be checked for its licence obligations. For example, React's MIT licence requires its copyright and permission notices to be retained. Hosted services such as Supabase, Cloudflare, and Gemini have separate terms governing account use, data processing, and service limits.

A future university deployment would require review of the applicable data-protection rules, including requirements for sending personal information to external providers. The UAE Government identifies Federal Decree-Law No. 45 of 2021 as part of the country's data-protection framework; its application would need to be checked for the institution and data involved.

Asset ownership, transfer approvals, donation, and disposal remain subject to university policies and delegated authority. The software can enforce agreed permissions and record decisions, but cannot grant authority that users do not hold.

Sources: UAE Government, ["Data protection laws"](#); [React MIT licence](#); applicable hosted-service terms.

6.2 Privacy, Security, and Responsible AI

Only information needed for the task should be sent to an AI service. Real personal or confidential data should not be used without institutional approval and appropriate processing arrangements. Gemini's terms distinguish unpaid usage, which may be used to improve products, from paid usage, which is not used for that purpose but may still involve limited logging.

The access controls and human-review steps in Section 4 remain essential. Testing should check whether recommendations favour particular departments without a justified reason and whether missing information produces misleading answers. Users must be able to correct suggestions, and important decisions must remain traceable to an authorized person.

Source: [Google Gemini API Additional Terms of Service](#).

7. Risk Assessment and Contingencies

These initial ratings reflect the proposed scope and dependencies and will be reviewed as development provides better evidence.

Risk	Likelihood / impact	Response and fallback
Schedule and integration delays	High / High	- Build an end-to-end exchange early; review progress weekly and reserve integration time. - If delayed, redistribute tasks and postpone optional refinements. Required scope changes need instructor agreement.

Incorrect AI results or service limits	Medium / High	• Test representative cases, validate responses, and monitor usage. • Keep conventional workflows available during failures; reassess the model or provider if necessary.
Unauthorized access or inconsistent records	Medium / High	• Test permissions, competing reservations, and restoration of records and files. • Block release of affected features until errors are corrected.
Low participation or outdated listings	Medium / High for deployment	• Keep updates simple, assign record ownership, and trial the workflow with participating departments. • Limit rollout if records cannot be maintained.
Costs exceeding benefits	Medium / High	• Compare actual effort and usage with estimates. • Reassess expansion if purchase avoidance is lower, or handling and support costs are higher, than assumed.

Each development task will have an owner and reviewer responsible for reporting related risks. Institutional process and data responsibilities would need named owners before a university pilot.

8. Project Schedule and Delivery Feasibility

The schedule follows the lab slides, with later dates subject to confirmation where the original brief differs.

Lab week, 2026	Deliverable and main work
21 September	Feasibility study and initial technical checks
5 October	Detailed requirements, permissions, test cases, and data definitions
26 October	Architecture and detailed design, informed by early integration trials
16 November	Working application with the required major features
23 November	Testing documentation, final report, presentation, and demonstration

Asset records and access controls must support the exchange workflow before AI is connected. Reporting depends on reliable records and agreed calculations. Technical trials should begin early, with testing and documentation continuing throughout development.

The 400-hour estimate across roughly nine weeks requires 44 team hours weekly, or 11 per member on average. With the 20% reserve, this rises to 53 team hours, or 13 per member. These are planning requirements, not confirmed commitments.

Delivery depends on confirming task ownership and availability, reviewing progress weekly, and integrating work regularly. The final week is reserved for corrections and demonstration preparation. Forecast delays should trigger task redistribution and a review of priorities.

9. Recommendation and Action Plan

The study recommends proceeding with the semester prototype, subject to the checks in Section 4 and confirmation of team time and service spending. The proposed managed services provide a credible technical approach, synthetic data removes the need for institutional database access, and estimated prototype service costs are modest.

A university deployment requires a separate decision. The financial scenario shows potential savings, but lower purchase avoidance produces a negative first-year return. Actual adoption, handling arrangements, data permissions, and support costs must therefore be assessed before wider use.

The next steps are to:

- Confirm task ownership and weekly availability.
- Test access controls, competing reservations, AI inputs, and backup recovery in a small working example.
- Refine requirements specifications and data definitions using those results, then develop the architecture and detailed design.
- Review progress, spending, and major risks at each milestone.

Sources

This report draws on the project brief, feasibility lab & lecture slides, finalized Phase 1 report, and the sources identified in the relevant sections. Provider documentation and pricing were checked in September 2026.